



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程设计实验报告

实验二：μC/OS-II 系统移植实验

院系： 计算机科学与技术学院

专业： 计算机科学与技术

姓名： 毛星博

学号： 2351520

指导老师： 郭玉臣

完成时间： 2026 年 6 月 15 日

一、实验目的

- 通过实现将 μ C/OS-II 系统移植到 Digilent NEXYS 4 DDR 开发板上，回顾编译原理、操作系统等课程的相关知识内容
- 贯通计算机相关课程如计算机组成原理、计算机系统结构、编译原理和操作系统等
- 为后续在系统上开发应用程序的实验打好基础

二、实验内容

参考雷思磊老师的《自己动手写 CPU》的思路，增加 Wishbone 总线，GPIO、UART、Flash 控制器、SDRAM 控制器。利用 Ubuntu 上建立交叉编译环境，对 μ C/OS-II 系统进行改写、编译。最终将 μ C/OS-II 系统移植到 Digilent NEXYS 4 DDR 开发板上并验证。

实验整体方案：

1. 增加 Wishbone 总线

- 为 OpenMIPS 增加 Wishbone 总线接口
- 增加 Wishbone 总线
- 增加 GPIO
- 增加 UART 控制器
- 增加 Flash 控制器
- 增加 SDRAM 控制器
- 实现完整的 SOPC

2. 操作系统 μ C/OS-II 的移植

- 在 Ubuntu 上建立交叉编译环境
- 对 μ C/OS-II 系统进行改写与编译

3. 下板验证检查

- 使用 SSCOM V5.13.1 串口/网络数据调试器，对结果进行检查

三、实验环境与硬件配置

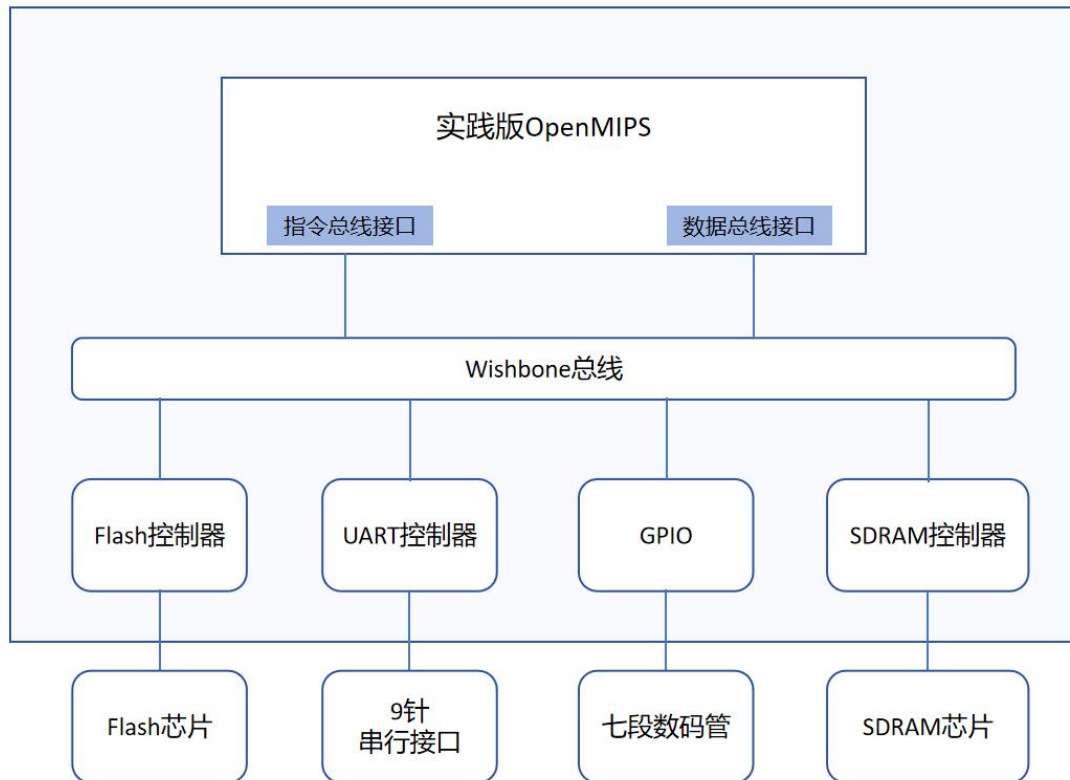
- 处理器：13th Gen Intel(R) Core(TM) i9-13900HX
- 开发环境：Vivado 2018.3
- 仿真环境：Vivado 2018.3
- 开发板：NEXYS4 DDR Atrix-7

四、系统设计总框架图

本次实验的实践班 OpenMIPS CPU 采用哈佛结构，将指令存储和数据存储分开。整体系统设计以实验版 OpenMIPS 处理器为核心，在处理器原有指令执行能力的基础上，进一步扩展片上系统结构，使其能够通过 Wishbone 总线访问多个外设模块和存储模块，最终形成一个能够运行 μ C/OS-II 实时操作系统的简化 SOPC 系统。

从系统设计总框架图可以看出，整个系统主要由 OpenMIPS 处理器、Wishbone 总线、Flash 控制器、UART 控制器、GPIO 模块以及 SDRAM 控制器组成。其中，OpenMIPS 处理器位于系统核心位置，负责取指、译码、执行指令以及发起对存储器和外设的访问请求。处理器通过指令总线接口和数据总线接口与 Wishbone 总线相连，Wishbone 总线则作为系统

内部各模块之间通信的统一通道。



五、实验步骤

5.1 实现 89 条 MIPS CPU

这部分是实验一中完成的内容，在此不在赘述。实验一中我们完成了 CPU 指令集的扩展工作，支持到 89 条指令。参考《自己动手写 CPU》，实现 CP0、异常处理，需要新增的 35 条指令如下：

- 移动操作指令： movn、movz
- 算术操作指令： clo、madd、maddu、msub、msubu
- 转移指令： b、bal、bgezal、bgtz、blez、bltz、bltzal
- 加载存储指令： ll、lwl、lwr、sc、swl、swr
- 异常相关指令： tge、tgeu、tlr、tlru、tne、teqi、tgei、tgeiu、tlr、tlru、tnei
- 其他指令： nop、ssnop、sync、pref

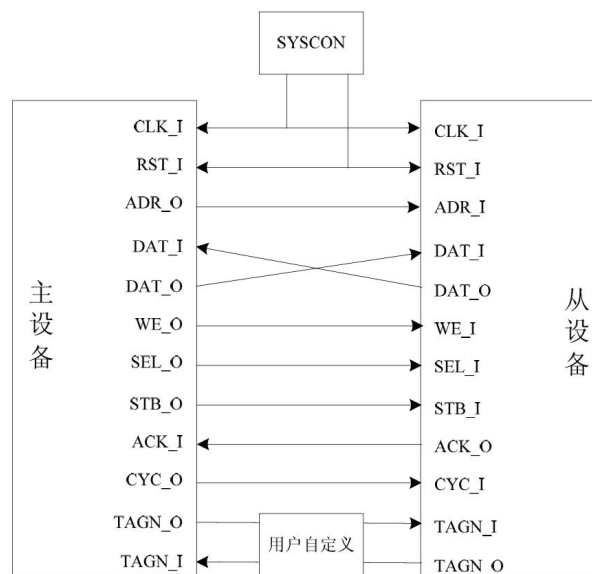
5.2 增加 Wishbone 总线

Wishbone 总线在本系统中起到“桥梁”和“互联枢纽”的作用。CPU 不需要直接面对每一个外设模块复杂的控制信号，而是通过统一的总线协议发起读写操作。无论是访问 Flash、UART、GPIO，还是访问 SDRAM，CPU 都可以通过地址、数据、读写控制信号和应答信号完成访问。这样设计的优点是系统结构清晰，模块之间耦合度较低，便于后续扩展和调试。

PC 机一般都提供 PCI 插槽，各种板卡（包括显卡、语音卡、网卡，甚至用户自制的板卡）只要满足 PCI 接口标准，就可以直接插在 PC 机的 PCI 插槽使用，十分方便。同样的道理，目前有很多 IP 核的研发者或公司，为了方便不同研发者或公司的 IP 核能够直接连接，就要求这些 IP 核遵守共同的接口标准。在片上系统（SoC）中，处理器核与其他 IP 核

通过共享总线互通互联，这些 IP 核必须遵守相同的总线规范。总线规范定义了 IP 核之间的通用接口。目前常见的片上总线规范有 ARM 公司的 AMBA、IBM 公司的 CoreConnect、Altera 公司的 Avalon，以及这里所说的 Wishbone。Wishbone 总线规范是 Silicore 公司最先提出的，由于其开放性，现在已有不少用户群，特别是一些免费的 IP 核，大多数都采用 Wishbone 规范。Wishbone 除了开放、免费，还有简单、灵活、轻量、支持用户自定义标签的特点。目前已有 B4 版本的规范，OR1200 中遵循的是 Wishbone B2 与 B3 版本的规范，用户可以自行配置是采用 B2 还是 B3 规范。

Wishbone 有多种互联方式：点对点、数据流、共享总线、交叉互联等。OR1200 内部使用的都是点对点连接方式。在点对点连接方式中，有一个主设备，一个从设备，连接关系如下图所示，图中输出信号使用“_O”结束，输入信号使用“_I”结束，同时所有的信号都是高电平有效。



图中主从设备接口的含义如下：

- (1) **CLK_I**、**RST_I**：分别是时钟信号、复位信号，由外部输入。
- (2) **DAT_O/DAT_I**：主设备和从设备之间的数据信号，数据可以由主设备传送给从设备，也可以由从设备传送给主设备。一对主设备和从设备之间最多存在两条数据总线，一条用于主设备向从设备传输数据，另一条用于从设备向主设备传输数据。
- (3) **ADR_O/ADR_I**：地址信号，主设备输出地址到从设备。
- (4) **WE_O/WE_I**：写使能信号，主设备输出到从设备，代表当前周期中进行的操作是写操作还是读操作，1 代表写，0 代表读。
- (5) **SEL_O/SEL_I**：数据总线选择信号，标识当前操作中数据总线上哪些比特是有效的，以总线粒度为单位。**SEL_O/SEL_I** 的宽度为数据总线宽度除以数据总线粒度。比如一个具有 32 位宽、粒度为 1 个字节的数据总线的选择信号应定义为 **SEL_O(3:0)/SEL_I(3:0)**，**SEL(4'b1001)** 代表当前操作中数据总线的最高和最低字节有效。
- (6) **CYC_O/CYC_I**：总线周期信号，**CYC_O/CYC_I** 有效代表一个主设备请求总线使用权或者正在占有总线，但是不一定正在进行总线操作（是否正在进行总线操作取决于选通信号 **STB_O/STB_I** 是否有效）。只有该信号有效，Wishbone 主设备和从设备之间的其它信号才有意义。**CYC_O/CYC_I** 信号在一次总线操作过程中必须持续有效，比如一次块读操作可能需要多个时钟周期，那么 **CYC_O/CYC_I** 信号必须在多个时钟周期中持续有效。

(7) STB_O/STB_I: 选通信号。选通有效代表主设备发起一次总线操作。只有选通信号有效时（此时 CYC_O/CYC_I 也必须为高），ADR_O/ADR_I、 DAT_O/DAT_I、 SEL_O/SEL_I 才有意义。

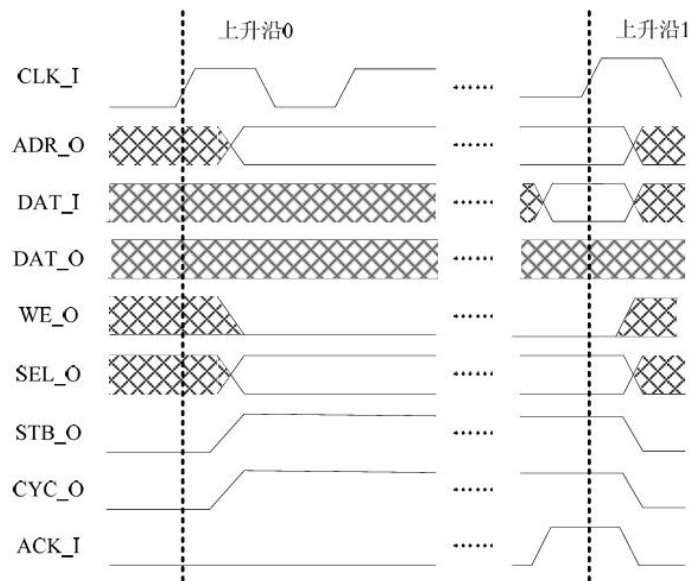
(8) ACK_O/ACK_I: 实际还可以有 ERR_O/ERR_I、 RTY_O/RTY_I，都表示主从设备间的操作结束方式信号。ACK 表示成功，ERR 表示错误， RTY 表示重试。操作总是在某一总线周期内完成的，因此操作结束方式也称为总线周期结束方式。成功是操作的正常结束方式，错误表示操作失败，造成失败的原因可能是地址或者数据校验错误，写操作或者读操作不支持等。重试表示从设备当前忙，不能及时处理该操作，可以稍后重新发起。接收到操作失败或者重试后，主设备如何响应取决于主设备的设计者。

(9) TAGN_O/TAGN_I: 标签信号，用户可以利用标签信号传递自定义的信息。

一个总线周期由多个不同的时钟周期构成，完成单次读/写操作、块读/写操作、读改写操作，总线周期也相应分为单次读/写周期、块读/写周期、读改写周期。

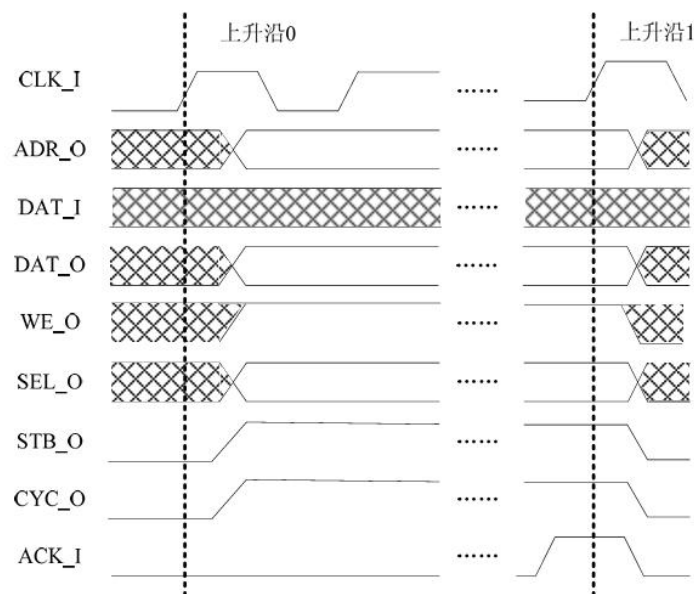
一般情况下，一次操作由主设备和从设备控制信号的一次握手，以及同时进行的地址和数据总线的一次传输构成。当主设备将 CYC_O 置高，一个总线周期开始，此后当 STB_O 为高时，一次总线操作开始。CYC_O 和 STB_O 可以同时从低电平变为高电平，表示发起总线周期的同时开始一次总线操作，因此在只有一个主设备时可以将两者合并为一个信号。在 OR1200 中就直接将 CYC_O、 STB_O 合并成 CYCSTB_O。

● 单次读操作的 Wishbone 总线信号



在时钟上升沿 0，主设备将地址信号 ADR_O、适当的 SEL_O 放到总线上，将 WE_O 置低表示读操作，将 CYC_O、STB_O 置高表示一次总线操作开始。在时钟上升沿 1 到达之前，从设备检测到主设备发起的操作，将适当的数据放到主设备的输入信号 DAT_I，同时将主设备的 ACK_I 置高作为对主设备 STB_O 的响应。从设备可以在 ACK_I 有效之前插入任意数量的等待状态。在时钟上升沿 1，主设备发现 ACK_I 信号为高，将 DAT_I 采样，并将 STB_O 和 CYC_O 置低表示操作完成。从设备发现 STB_O 置低后，将主设备的输入信号 ACK_I 也置低。单次读操作就完成了。

● 单次写操作的 Wishbone 总线信号



在时钟上升沿 0，主设备将地址信号 ADR_O、数据信号 DAT_O 放到总线上，将 WE_O 置高表示写操作，将适当的 SEL_O 放到总线上指示从设备 DAT_O 中哪些字节是有效的，将 CYC_O、STB_O 置高表示一次总线操作开始。

在时钟上升沿 1 到达之前，从设备检测到主设备发起的操作，将锁存 DAT_O 的数据，同时将主设备的 ACK_I 置高作为对主设备 STB_O 的响应。从设备可以在 ACK_I 有效之前插入任意数量的等待状态。

在时钟上升沿 1，主设备发现 ACK_I 信号为高，将 STB_O 和 CYC_O 置低表示操作完成。从设备发现 STB_O 置低后，将主设备的输入信号 ACK_I 也置低。单次写操作就完成了。

5.2.1 Wishbone 总线接口模块 (wishbone_bus_if)

为了让处理器能够通过 Wishbone 总线访问多个外设，需要在处理器与总线之间插入 Wishbone 总线接口模块 (wishbone_bus_if)，将 CPU 内部的访存请求转换为符合 Wishbone 协议的总线事务。

```
module wishbone_bus_if(
    input wire      clk,
    input wire      rst,

    // 来自 ctrl 模块
    input wire[5:0]  stall_i,
    input wire       flush_i,    // 处理异常，需要清空流水线

    // CPU 侧的接口
    input wire       cpu_ce_i,   // 来自处理器的访问请求信号
    input wire[RegBus] cpu_data_i, // 来自处理器的数据
    input wire[RegBus] cpu_addr_i,
    input wire       cpu_we_i,
    input wire[3:0]  cpu_sel_i,
    output reg[RegBus] cpu_data_o, // 输出到处理器的数据

```

```

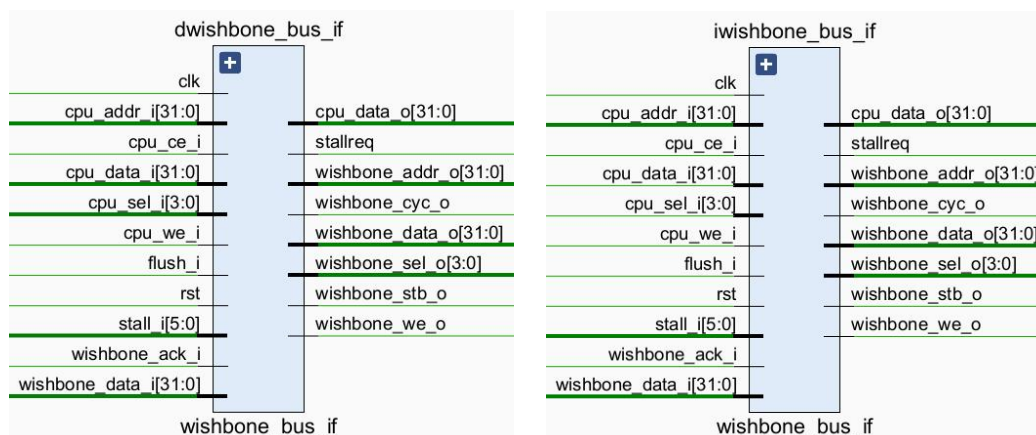
// Wishbone 侧的接口
input wire[`RegBus]      wishbone_data_i, // Wishbone 总线输入的数据
input wire               wishbone_ack_i,
output reg[`RegBus]      wishbone_addr_o,
output reg[`RegBus]      wishbone_data_o,
output reg               wishbone_we_o,
output reg[3:0]          wishbone_sel_o,
output reg               wishbone_stb_o, // Wishbone 总线选通信号
output reg               wishbone_cyc_o, // Wishbone 总线周期信号

output reg               stallreq
);

```

由于 OpenMIPS 采用哈佛结构，指令通道和数据通道是独立的，因此需要实例化 两个 Wishbone 总线接口模块：

- 指令 Wishbone 总线接口 (iwishbone_bus_if)：连接取指阶段 (IF)，将 PC 寄存器输出的指令地址转换为 Wishbone 总线读请求，取回的指令数据送入流水线。
- 数据 Wishbone 总线接口 (dwishbone_bus_if)：连接访存阶段 (MEM)，将 load/store 指令的地址和数据转换为 Wishbone 总线读/写请求。



wishbone_bus_if 模块的核心是一个三状态有限状态机，用于管理 Wishbone 总线事务的生命周期。

(1) WB_IDLE (空闲状态)

当 CPU 侧没有有效的访存请求 ($\text{cpu_ce_i} = 0$)，或流水线正在被清除 ($\text{flush_i} = 1$) 时，状态机保持空闲。一旦检测到 $\text{cpu_ce_i} = 1$ 且 $\text{flush_i} = 0$ ，立即将 CPU 的地址、数据、读写控制、字节选择信号加载到 Wishbone 总线输出端口，置位 stb_o 和 cyc_o 发起总线事务，同时向流水线控制器发出暂停请求 ($\text{stallreq} = \text{Stop}$)，防止后续指令覆盖当前请求。状态转入 WB_BUSY。

(2) WB_BUSY (总线忙状态)

在此状态下等待从设备返回应答信号 wishbone_ack_i 。分为以下几种情况：

- 收到 ack_i ：总线事务完成。清除 stb_o 、 cyc_o 等总线信号。若为读操作，将从设备返回的数据 wishbone_data_i 缓存到 rd_buf 寄存器，同时直接输出到 cpu_data_o 。撤销暂停请求。若此时流水线仍处于暂停 ($\text{stall_i} \neq 0$)，则进入 WB_WAIT_FOR_STALL 状态；否则直接返回 WB_IDLE。
- 收到 flush_i ：异常/中断发生，需要立即中止总线事务。清除所有总线信号和缓冲数据，

直接返回 WB_IDLE。

- 两者都未收到：继续等待，保持暂停请求。

(3) WB_WAIT_FOR_STALL（等待暂停解除状态）

此状态用于处理一种特殊场景：总线事务已经完成（收到了 ack），但流水线因其他原因（如除法运算、其他总线接口占用等）仍处于暂停状态。在此状态下，cpu_data_o 输出缓存在 rd_buf 中的数据，不发出暂停请求。当 stall_i 全部为 0 时返回 WB_IDLE。

5.2.2 在 OpenMIPS 中实例化 Wishbone 总线接口

在 OpenMIPS 处理器顶层模块 openmips.v 中，实例化了两个 wishbone_bus_if 模块，分别对应指令通道和数据通道：

```
wishbone_bus_if dwishbone_bus_if(  
    .clk(clk),  
    .rst(rst),  
  
    // 来自控制模块 ctrl  
    .stall_i(stall),  
    .flush_i(flush),  
  
    // CPU 侧读写操作信息  
    .cpu_ce_i(ram_ce_o),  
    .cpu_data_i(ram_data_o),    // 来自 CPU 的数据  
    .cpu_addr_i(ram_addr_o),  
    .cpu_we_i(ram_we_o),  
    .cpu_sel_i(ram_sel_o),  
    .cpu_data_o(ram_data_i),  
  
    // Wishbone 总线侧接口  
    .wishbone_data_i(dwishbone_data_i),  
    .wishbone_ack_i(dwishbone_ack_i),  
    .wishbone_addr_o(dwishbone_addr_o),  
    .wishbone_data_o(dwishbone_data_o),  
    .wishbone_we_o(dwishbone_we_o),  
    .wishbone_sel_o(dwishbone_sel_o),  
    .wishbone_stb_o(dwishbone_stb_o),  
    .wishbone_cyc_o(dwishbone_cyc_o),  
  
    .stallreq(stallreq_from_mem)  
);  
  
wishbone_bus_if iwishbone_bus_if(  
    .clk(clk),  
    .rst(rst),  
  
    // 来自控制模块 ctrl  
    .stall_i(stall),
```

```

.flush_i(flush),

// CPU 侧读写操作信息
.cpu_ce_i(rom_ce),
.cpu_data_i(32'h00000000),
.cpu_addr_i(pc),
.cpu_we_i(1'b0),
.cpu_sel_i(4'b1111),
.cpu_data_o(inst_i),

// Wishbone 总线侧接口
.wishbone_data_i(iwishbone_data_i),
.wishbone_ack_i(iwishbone_ack_i),
.wishbone_addr_o(iwishbone_addr_o),
.wishbone_data_o(iwishbone_data_o),
.wishbone_we_o(iwishbone_we_o),
.wishbone_sel_o(iwishbone_sel_o),
.wishbone_stb_o(iwishbone_stb_o),
.wishbone_cyc_o(iwishbone_cyc_o),
.stallreq(stallreq_from_if)
);

```

5.2.3 增加 Wishbone 总线互连模块

本系统采用开源的 WB_CONMAX (Wishbone Connection Matrix) IP 核作为总线互连模块。WB_CONMAX 来自 OpenCores 社区，由 Rudolf Usselman 开发，是一个支持多主多从的共享总线互连矩阵，支持最多 8 个主设备和 16 个从设备的连接。

在顶层 SOPC 模块 openmips_min_sopc.v 中，WB_CONMAX 的主/从端口分配如下：
主设备端口分配：

端口编号	连接模块	说明
Master 0	OpenMIPS 数据 Wishbone 接口	负责 load/store 指令的总线访问
Master 1	OpenMIPS 指令 Wishbone 接口	负责取指令的总线访问
Master 2~7	未使用	信号置为无效 (全 0)

从设备端口分配：

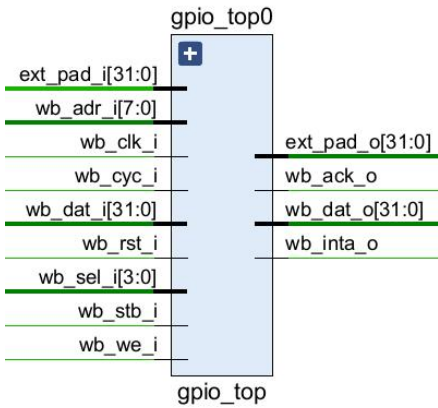
端口编号	连接模块	地址范围 (addr[31:28])	说明
Slave 0	SDRAM 控制器 (DDR2)	0x0---_----	程序与数据存储
Slave 1	UART 控制器	0x1---_----	串口通信
Slave 2	GPIO 模块	0x2---_----	通用输入输出
Slave 3	Flash 控制器	0x3---_----	存储引导程序

5.3 增加 GPIO 模块

GPIO (General Purpose Input Output, 通用输入/输出) 是以位 (bit) 为单位进行数字信号输入和输出的 I/O 接口。作为纯粹的通用输入/输出通道，它在输入状态下从外部读取信

号，在输出状态下则将写入的数值输出到外部。通过 GPIO，处理器能够与多种外设进行连接与通信，例如 LED、开关、七段数码管等（本次实验 GPIO 将控制七段数码管）。

在本次实验中，我们使用的是《自己动手写 CPU》的附件中提供的 OpenCores 网站的开源项目 GPIO IP Core（GPIO IP 核），并将其添加和整合到了整个系统之中。

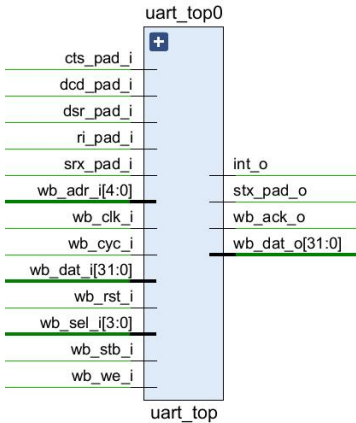


GPIO 中的主要寄存器如下：

寄存器名称	地址	宽度	访问方式	作用描述
RGPIO_IN	Base + 0x0	1~32	只读	输入到 GPIO 的信号
RGPIO_OUT	Base + 0x4	1~32	可读可写	GPIO 输出的信号
RGPIO_OE	Base + 0x8	1~32	可读可写	GPIO 输出接口使能信号
RGPIO_INTE	Base + 0xC	1~32	可读可写	中断使能信号

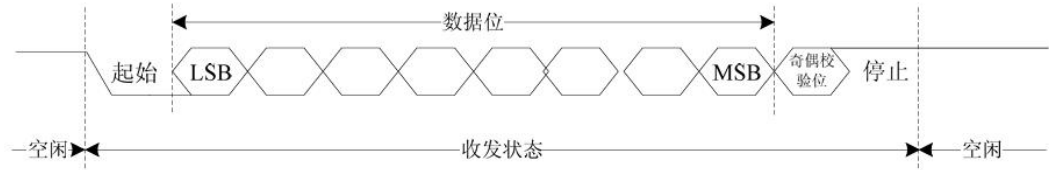
5.4 增加 UART 控制器

在 OpenMIPS 处理器的设计实践中，所采用的 UART 控制器同样是《自己动手写 CPU》的附件中提供的源自 OpenCores 网站的开源项目 UART16550 IP Core（其源代码可在 OpenCores 平台获取）。通过在系统中添加并整合该 UART 控制器，即可实现系统与计算机之间的串口通信。



UART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）是一种应用非常广泛的串行数据传输协议。它的核心功能是实现并行数据与串行数据之间的相互转换：在发

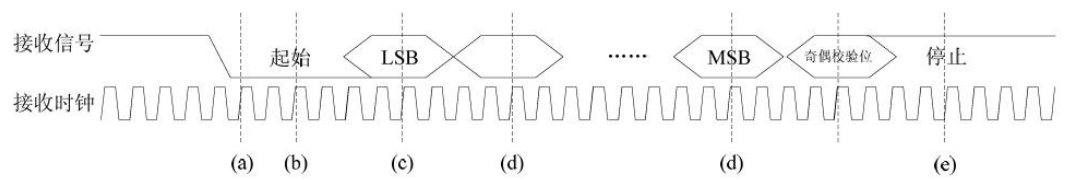
送时将并行数据转为串行数据输出，在接收时则将串行数据还原为并行数据。在传输过程中，UART 会将待传数据的每个字符按位（bit）逐一进行传送。其典型的传输格式依次包括：起始位、数据位、奇偶校验位和停止位，具体说明如下：



- 起始位：首先发出一个低电平信号（逻辑“0”），以此作为一帧数据传输开始的标识。
- 数据位：紧随起始位之后。数据位的长度可以根据协议配置为 4、5、6、7、8 位等，它们共同构成一个字符，传输时从数据的最低位（LSB）开始依次传送。
- 奇偶校验：位于数据位之后。通过在数据位后附加这一位，使整个数据帧中“1”的个数呈现为偶数（偶校验）或奇数（奇校验），接收端以此来初步校验数据传输的正确性。
- 停止位：作为字符数据传输结束的标志（通常为高电平）。

UART 的通信速率通过波特率（baud rate）来衡量。波特率是指信号调制后的变化率，即单位时间内载波变化的次数。由于波特率的计算对象除了数据位之外，还包含了起始位、校验位和停止位等辅助控制位，因此波特率与纯粹的数据传输速率是有所区别的。在实际应用中，UART 常用的波特率有 9600 baud、19200 baud、38400 baud 等。本实验中，我们采用 9600 的波特率。

在接收端，UART 采用高于波特率的采样频率来确保数据接收的准确性。在实际设计中，通常会使用 16 倍于波特率的接收时钟进行过采样。为了便于理解，以下以 4 倍于波特率的接收时钟为例，说明具体的数据接收过程：

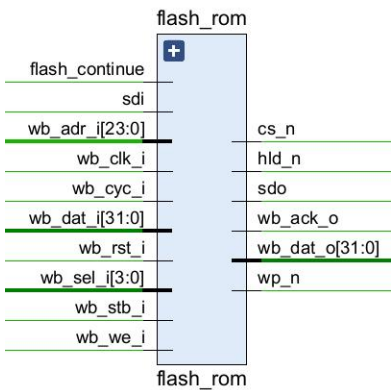


1. 起始位检测：当接收信号由高电平跃变为低电平时，表明系统检测到了可能的起始位信号。
2. 起始位确认：在检测到起始位后，系统在接下来的第 2 个时钟周期对信号进行再次检查。若此时仍保持低电平，则判定为真实有效的起始位，并正式启动数据接收；若变为高电平，则视其为干扰或噪声，予以忽略。
3. 数据位采样（LSB）：确认起始位有效后，等待 4 个时钟周期再次采样接收信号，此时得到的值即为传入的第一个数据位（最低位 LSB）。
4. 后续位采样：此后，系统每隔 4 个时钟周期对信号进行一次采样，从而依次获取后续的数据位和奇偶校验位。这种采样机制能够确保每次采样点都落在每个数据位的正中间，从而极大提高了数据读取的准确性与稳定性。
5. 结束与应用：在数据接收完成后，接收停止位。

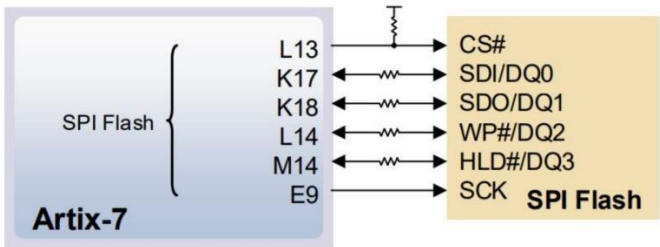
5.5 增加 Flash 控制器

在操作系统移植的过程中，需要使用非易失性存储介质来存放引导程序（BootLoader）以及操作系统的二进制镜像文件，Flash 芯片在系统中正承担着这一角色。与《自己动手写

CPU》一书中所实现的传统并行 Flash 不同，本次实验使用的是 Nexys4 DDR 开发板上板载的 SPI Flash 模块。



在实践版 OpenMIPS 的设计中，实现了一个单次能够读取 4 字节（4 Bytes）数据的 Flash 控制器。该控制器主要用于模拟“硬盘”的功能，负责读取包括 BootLoader 和操作系统二进制文件在内的所有只读数据。Nexys4 DDR 开发板所采用的 SPI Flash 芯片具体型号为 S25FL128S。



各接口的具体功能和说明如下：

PAD NO.	PAD NAME	I/O	FUNCTION
1	/CS	I	Chip Select Input
2	DO (IO1)	I/O	Data Output (Data Input Output 1)* ¹
3	/WP (IO2)	I/O	Write Protect Input (Data Input Output 2)* ²
4	GND		Ground
5	DI (IO0)	I/O	Data Input (Data Input Output 0)* ¹
6	CLK	I	Serial Clock Input
7	/HOLD (IO3)	I/O	Hold Input (Data Input Output 3)* ²
8	VCC		Power Supply

*1: IO0 and IO1 are used for Standard and Dual SPI instructions

*2: IO0 – IO3 are used for Quad SPI instructions

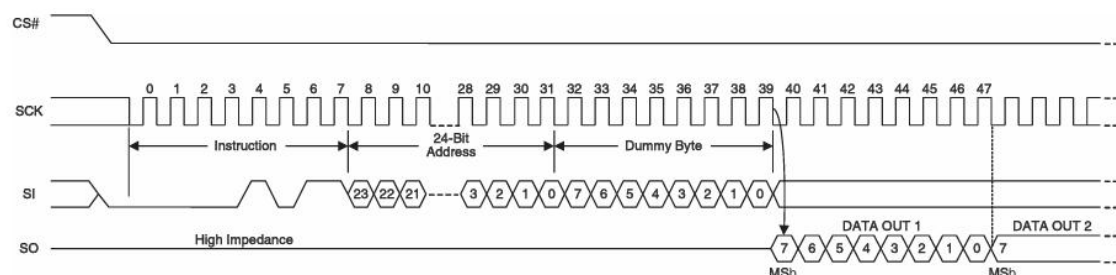
SPI 协议是一种常见的串行总线协议，其特点是数据在单根信号线上按位（bit）进行逐位传输。在本次设计中，主要使用以下四根信号线：

- SCK（时钟信号）：用于信号同步的时钟线，其频率可在芯片规定的工作范围内自由选择。
- CS_N（片选信号）：低电平有效（“_n”代表低电平有效）。只有当 CS_N 被拉低时，该 Flash 芯片才会被使能并开始工作。
- SI（串行输出/输入至主设备）：在本设计中，SI 是输入到主控设备（CPU 端）的信号线，Flash 芯片读取并向外输出数据时需通过该引脚进行传输。
- SO（串行输入/输入至 Flash）：在本设计中，SO 是输入到 Flash 芯片的信号线，向 Flash 写入指令或地址时需通过该引脚进行传输。

Flash 芯片本质上是由可读写的存储颗粒（EEPROM 介质）和一个内部控制器组成的，该控制器封装了许多用于操作存储颗粒的指令。因此，使用 Flash 芯片的关键在于向其内

部的控制器发送正确的操作指令。

由于本次实验仅需要实现数据的读取，因此主要涉及 **READ** 这条指令。根据 **S25FL128S** 的官方技术文档，读取指令的具体操作时序与步骤如下：



1. 使能芯片：首先将片选信号 **CS_N** 拉低，使能 **Flash** 芯片。
2. 配置时钟：使能后保持时钟信号 **SCK** 在工作范围内。查阅技术文档可知，**READ** 指令的最大工作频率为 **50MHz**。由于 **Nexys4 DDR** 开发板的板载晶振频率为 **100MHz**，因此选择 **50MHz** 作为 **SCK** 的工作频率，能够非常方便地通过 2 分频电路来实现。
3. 发送命令与地址：通过 **SO** 信号线，按位 (bit) 依次输入 **READ** 操作码 (**03H**) 以及 24 位的目标读取地址。
4. 接收数据：命令和地址发送完毕后，**Flash** 芯片会通过 **SI** 信号线，将目标地址中的数据逐位 (bit) 输出给主设备。

5.6 增加 SDRAM 控制器

SDRAM (Synchronous Dynamic Random Access Memory，同步动态随机访问存储器) 是一种常见的半导体存储器。其中：

- 同步：指其工作需要同步时钟的配合，内部命令的发送与数据的传输均以该时钟为基准；
- 动态：指存储阵列中的电容电荷会随时间泄漏，因此需要不断进行刷新 (Refresh) 操作以保证数据不丢失；
- 随机访问：指数据无需像磁带那样线性依次读写，而是可以自由指定任意地址进行即时的读写操作。

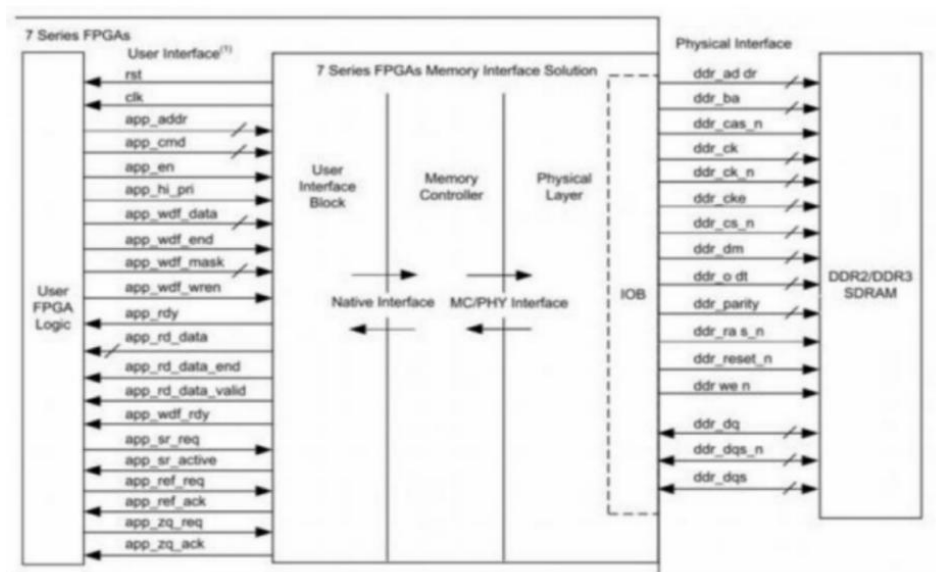
与《自己动手写 CPU》书中所实现的传统 **SDRAM** 不同，本次实验使用的是 **Nexys4 DDR** 开发板上板载的 **DDR2** 模块。在此次实验中，我们使用该 **DDR2** 来模拟一块最小读写宽度为 **8 bits**、最大读写宽度为 **32 bits** 的内存空间。

Nexys4 DDR 开发板上搭载的 **DDR2 SDRAM** 芯片型号为 **MT47H64M16HR-25:H**。由于 **DDR2** 协议本身非常复杂，**Xilinx** 官方提供了一个用于简化 **DDR** 内存控制的 IP 核——**Memory Interface Generator (MIG)**。在本次实验中，我们即通过该 IP 核来对内存进行操作。**MIG** 控制接口在用户端主要提供以下交互信号：

- **app_addr [26:0]** (输入)：数据读写的目标地址。
- **app_cmd [2:0]** (输入)：操作指令。其中 **3'b001** 代表读操作，**3'b000** 代表写操作。
- **app_en** (输入)：指令使能信号。
- **app_rdy** (输出)：指令接收就绪信号。当 **SDRAM (MIG)** 成功接收到操作信号后，会将该信号拉高。
- **app_wdf_data [127:0]** (输入)：写入的数据。数据在通过掩码处理后写入 **SDRAM**。
- **app_wdf_mask [15:0]** (输入)：写数据字节掩码。该信号的 1 位 (bit) 对应写数据中的 1 字节 (Byte)。将某一位拉高意味着在写入 **SDRAM** 时，屏蔽 (忽略) 该位所对应的那个数据字节。
- **app_wdf_end** (输入)：写入结束信号。在发送最后一个写数据字时将其拉高，通知

SDRAM 写入操作结束。

- **app_wdf_wren**（输入）：写数据使能信号。当准备好 **app_wdf_data** 和 **app_wdf_mask** 后将该信号拉高，通知 SDRAM 开始写入数据。
- **app_wdf_rdy**（输出）：写数据接收就绪信号。当 SDRAM 成功接收到写入数据与掩码后会将其拉高。
- **app_rd_data [127:0]**（输出）：读取出的数据。
- **app_rd_data_valid**（输出）：读取数据有效标志。当 SDRAM 成功将读取的数据输出到 **app_rd_data** 线上时，该信号会被拉高。
- **app_sr_req**（输入）：保留引脚，实验中应将其拉低。
- **app_ref_req**（输入）：刷新请求信号。拉高该信号（需保持一个时钟周期）可向 DRAM 发送一条刷新指令。
- **app_zq_req**（输入）：ZQ 校准请求信号。拉高该信号（需保持一个时钟周期）可向 DRAM 发送 ZQ 校准指令，用于校准其片上终端匹配电阻（ODT）的阻值。
- **app_sr_active**（输出）：保留引脚。
- **app_ref_ack**（输出）：刷新响应信号。SDRAM 响应刷新请求时会将其拉高。
- **app_zq_ack**（输出）：校准响应信号。SDRAM 响应 ZQ 校准请求时会将其拉高。
- **ui_clk**（输出）：MIG IP 核提供给用户逻辑的工作时钟。所有与该 IP 核交互的时序逻辑电路都必须同步于此主时钟。
- **ui_clk_sync_rst**（输出）：与 **ui_clk** 同步的复位信号，其使用原则与上述时钟相同。



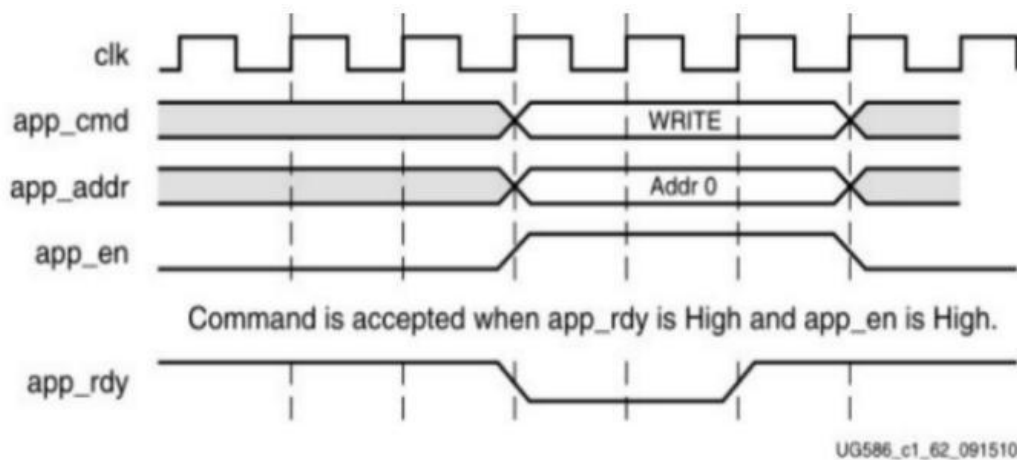
该 DDR2 芯片是 Nexys4 DDR 板卡上的重要硬件资源，容量大小为 128MB。因为 DDR 的底层硬件控制极其繁杂，即便借助 MIG IP 核封装了物理层信号（使得用户不必直接面对复杂的物理层波形，而只需关注应用层信号），其控制逻辑依然较为繁琐，需要进行多级封装。此外，相比于 CPU 的运行速度，DDR 属于相对慢速的设备，因此需要实现一个专门的 MIG 控制器来协调 CPU 与 DDR 之间的数据交互。

当 MIG IP 核生成后，我们需要将 DDR 的物理接口引出并添加到顶层模块中。值得注意的是，这些引脚不需要我们在 XDC 约束文件中手动进行端口配置，因为在 IP 核的生成和配置过程中，Vivado 已经自动完成了约束的绑定。

FPGA 与 DDR 之间的读写交互主要依赖于控制器底层的以下三种核心时序：

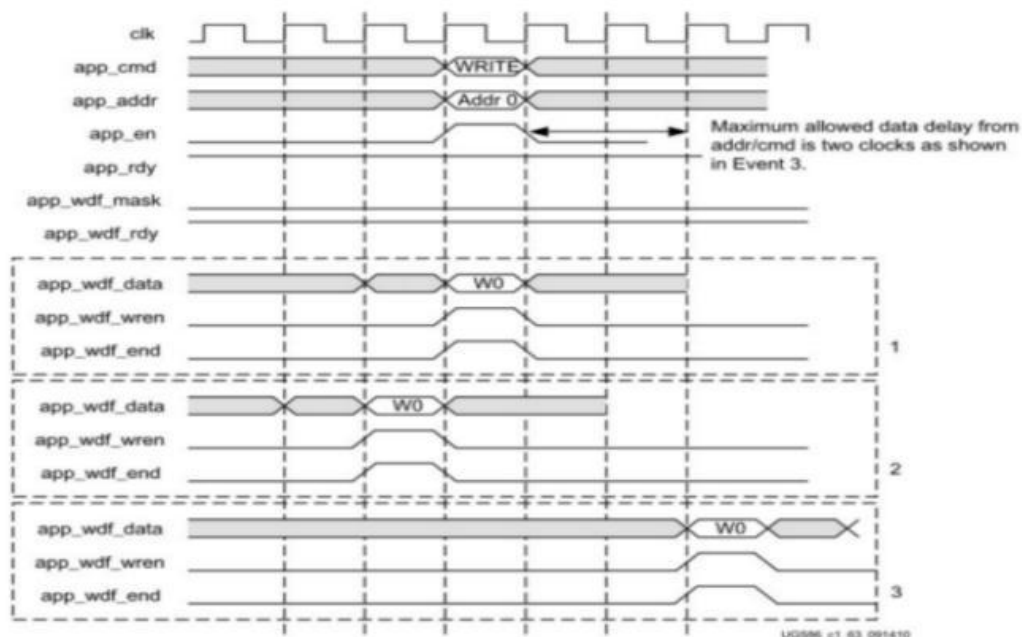
1. 控制信号时序：

在发出任何读写命令时，程序必须实时检测 **app_rdy** 信号。只有在 **app_rdy** 有效时，控制器才会接收该读写命令。



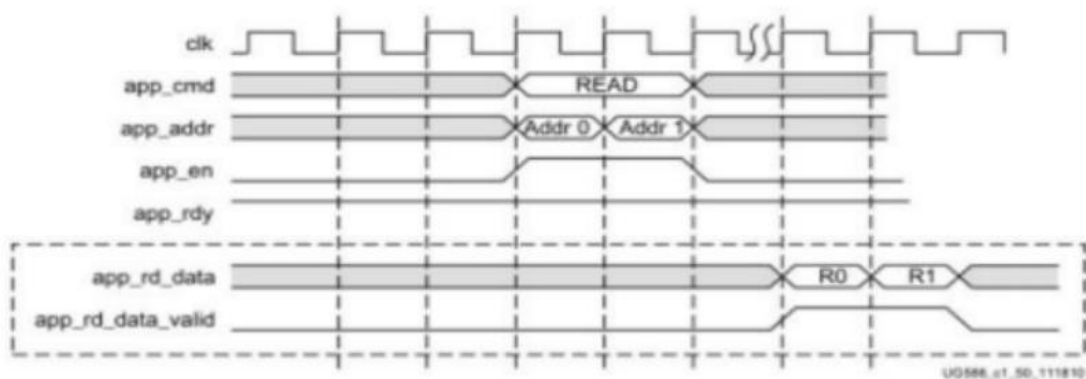
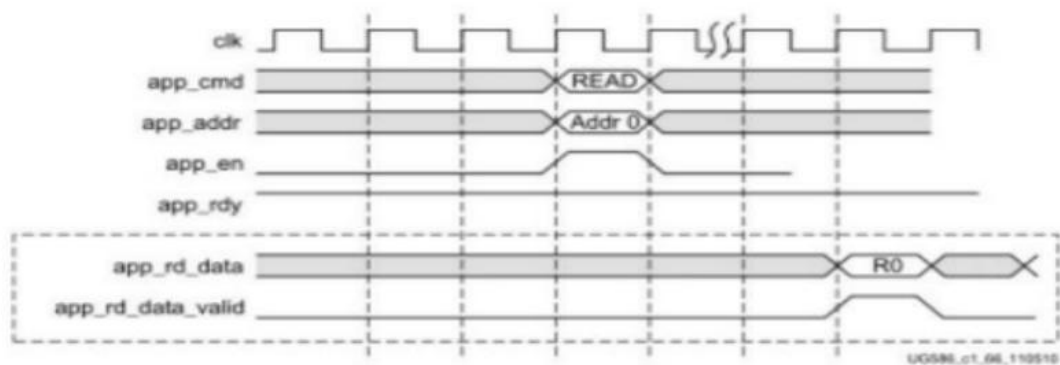
2. 写操作时序：

在执行写操作时，需要同时提供写命令 **app_cmd**、目标地址 **app_addr**、写入数据 **app_wdf_data** 等信号。写入数据的有效窗口相对宽裕：其最多可以比写命令提前一个时钟周期有效，最迟也可以比写命令晚两个时钟周期有效。必须注意的是：在写数据时，需检测并确保 **app_rdy** 和 **app_wdf_rdy** 信号同时有效。否则，写入命令将无法被正确送入 DDR 控制器的命令 FIFO 中，从而导致写操作失败。



3. 读操作时序：

读操作的时序相对简单。在发出读命令后，用户只需关注并检测 **app_rdy** 信号是否有效即可。



5.7 编写顶层模块 openmips_min_sopc.v

将 OpenMIPS 处理器、Wishbone 总线互联 1 矩阵、GPIO 模块、UART 控制器、Flash 控制器、SDRAM 控制器连接起来。

openmips_min_sopc.v 的接口定义如下：

```
module openmips_min_sopc(
    input wire      clk_in,
    input wire      rst_n,
    input wire      flash_continue,

    // UART 串口
    input wire      uart_in,
    output wire     uart_out,

    // 16 位外部输入，送入 GPIO 控制器
    input wire[15:0] gpio_i,

    // 32 位外部输出：
    // [31:16] LED, [15:8] 数码管段选, [7:0] 数码管位选
    output wire[31:0] gpio_o,

    // Flash 接口
    output cs_n,
    input  sdi,
```

```

output sdo,
output wp_n,
output hld_n,

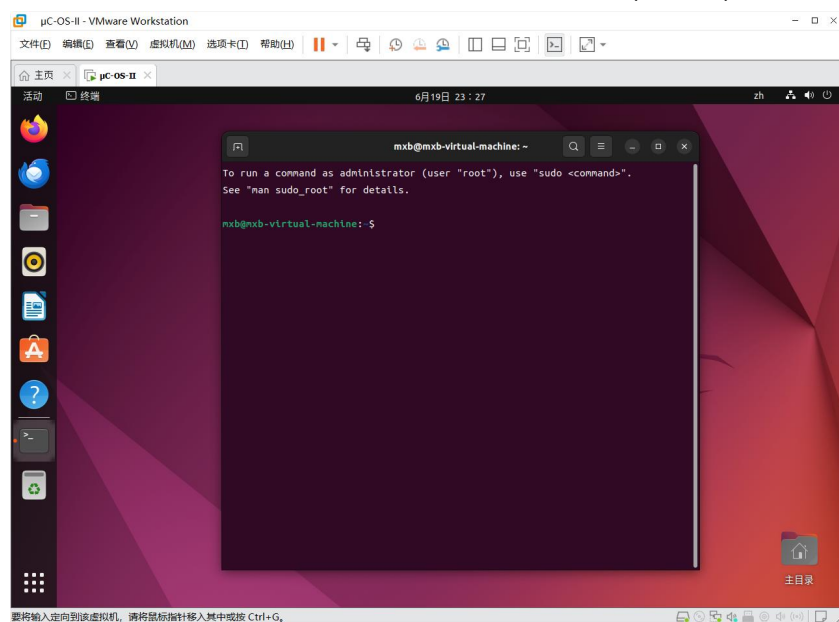
// DDR2 接口
inout [15:0]      ddr2_dq,
inout [1:0]      ddr2_dqs_n,
inout [1:0]      ddr2_dqs_p,
output [12:0]     ddr2_addr,
output [2:0]     ddr2_ba,
output           ddr2_ras_n,
output           ddr2_cas_n,
output           ddr2_we_n,
output [0:0]     ddr2_ck_p,
output [0:0]     ddr2_ck_n,
output [0:0]     ddr2_cke,
output [0:0]     ddr2_cs_n,
output [1:0]     ddr2_dm,
output [0:0]     ddr2_odt
);

```

5.8 利用 Ubuntu 上建立交叉编译环境

5.8.1 Linux-Ubuntu 环境配置

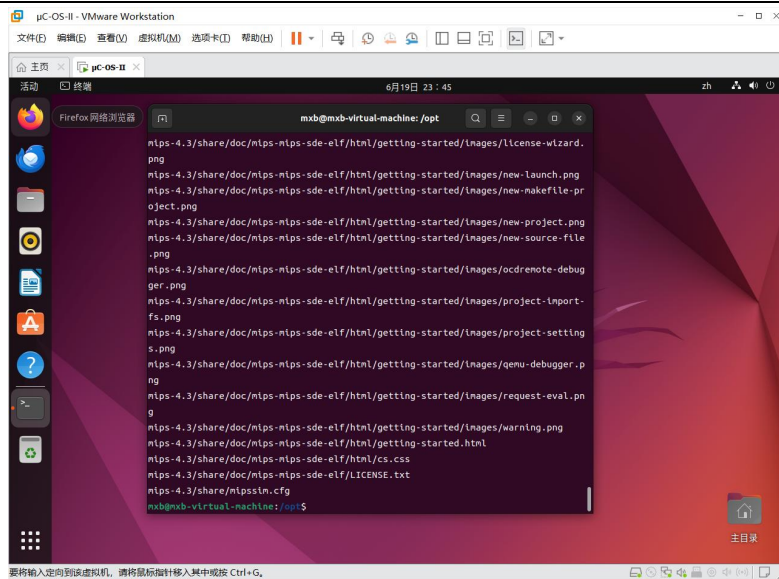
这里我是用 VMware 虚拟机，在其中安装 Linux 系统，使用 Ubuntu 的光盘映像文件进行安装；参考的是 CSDN 教程：[VMware 虚拟机安装 Ubuntu 教程\(超详细\)](#)



5.8.2 MIPS 编译环境的建立

首先需要安装 GNU 工具链。《自己动手写 CPU》一书的附件中提供了安装文件，mips-sde-elf-i686-pc-linux-gnu.tar.tar，将安装文件复制到 Ubuntu 的 /opt 目录下，打开 Ubuntu 的终端，使用如下命令解压缩：

```
cd /opt  
tar vfxj mips-sde-elf-i686-pc-linux-gnu.tar.tar
```



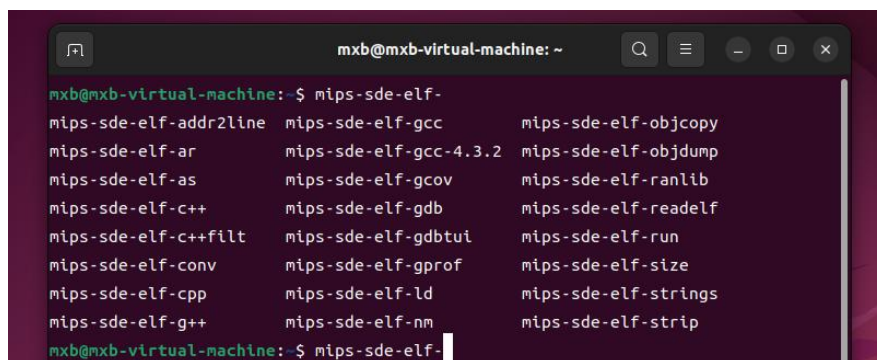
然后打开用户主目录 **Home** 文件夹，在窗口菜单栏中选择 **View->Show Hidden Files**，以显示所有文件，这样可以找到一个隐藏文件 **.bashrc**，在此文件的最后加入 **PATH** 的设置，如下。

```
export PATH="$PATH:/opt/mips-4.3/bin"
```

还需要安装 **libc32z1**、**libc6-dev-i386** 库

```
sudo dpkg --add-architecture i386  
  
sudo apt update  
  
sudo apt install libc6:i386 libstdc++6:i386 zlib1g:i386 libncurses5:i386 -y
```

重新启动 **Ubuntu** 系统。重启后，打开终端，在其中输入 **mips-sde-elf**，然后按两次 **Tab** 键，会列出刚刚安装的，针对 **MIPS** 平台的所有编译工具，如下图所示，表示 **GNU** 工具链安装成功。



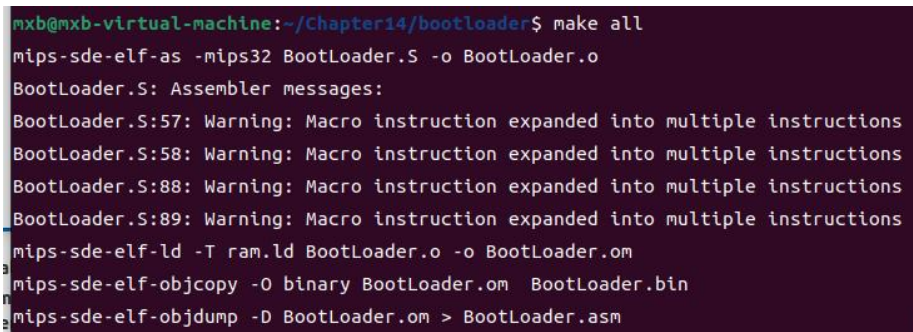
5.9 对 μ C/OS-II 系统进行改写、编译

首先需要修改 **BootLoader** 程序并重新编译，主要是修改 **UART** 的频率。修改如下：



```
1 .set noat
2 .set noreorder
3 .set nomacro
4
5 .org 0x0
6 .text
7 .align 4
8 .global _start
9 _start:
10
11 lui $1,0x1000
12 ori $1,$1,0x0003
13 ori $2,$0,0x80
14 sb $2,0x0($1)
15
16 lui $1,0x1000
17 ori $1,$1,0x0001
18 ori $2,$0,0x02
19 sb $2,0x0($1) # MSB of divisor latch
20
21 lui $1,0x1000
22 ori $1,$1,0x0000
23 ori $2,$0,0x8B
24 sb $2,0x0($1) # LSB of divisor latch
25
26 lui $1,0x1000
27 ori $1,$1,0x0003
28 ori $2,$0,0x03
29 sb $2,0x0($1) # 8bit, no parity, 1 stop bit
30
31 lui $1,0x2000
32 ori $1,$1,0x0008
33 lui $2,0xffff
34 ori $2,$2,0xffff
35 sw $2,0x0($1)
36
37 lui $1,0x2000
```

执行 `make all` 对 BootLoader 进行编译，生成可执行对象文件 `BootLoader.o`，将其转为可执行的汇编代码 `BootLoader.asm`（在 `makeall` 中已实现）



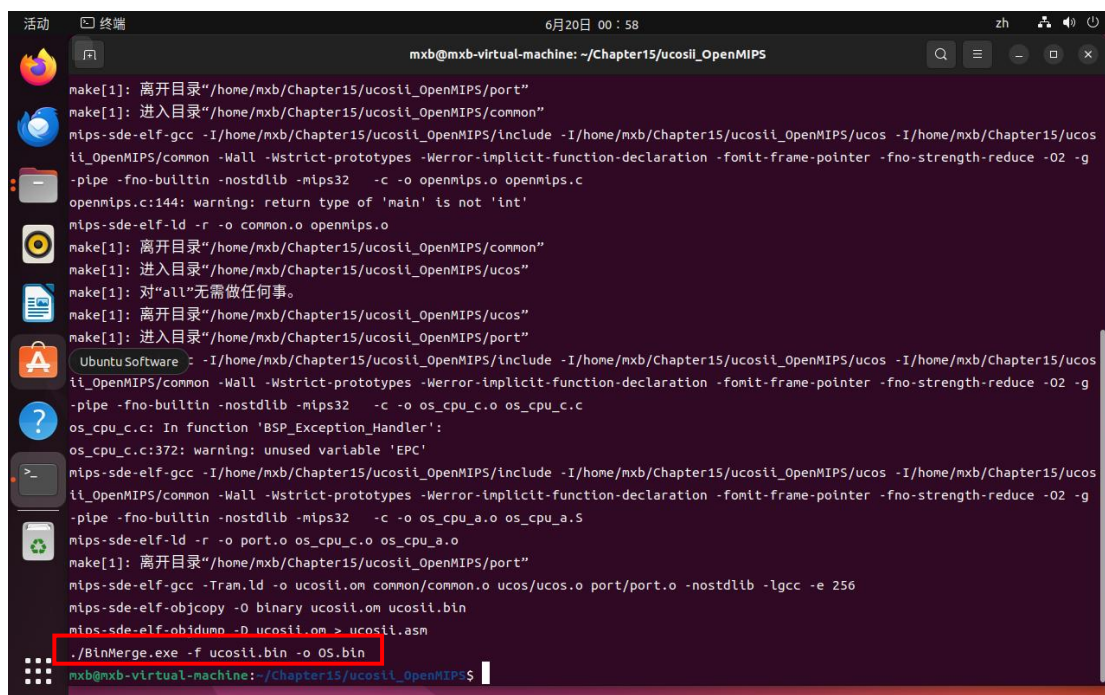
```
mxh@mxh-virtual-machine:~/Chapter14/bootloader$ make all
mips-sde-elf-as -mips32 BootLoader.S -o BootLoader.o
BootLoader.S: Assembler messages:
BootLoader.S:57: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:58: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:88: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:89: Warning: Macro instruction expanded into multiple instructions
mips-sde-elf-ld -T ram.ld BootLoader.o -o BootLoader.om
mips-sde-elf-objcopy -O binary BootLoader.om BootLoader.bin
mips-sde-elf-objdump -D BootLoader.om > BootLoader.asm
```

然后进行 μ C/OS-II 系统的编译，主要用的是《自己动手写 CPU》中提供的系统，在此基础上修改 UART 频率，修改如下：



```
1 #define REG8(add) *((volatile INT8U *) (add))
2 #define REG16(add) *((volatile INT16U *) (add))
3 #define REG32(add) *((volatile INT32U *) (add))
4
5 #define IN_CLK 100000000 /* 输入时钟是27MHz */
6
7 /*****
8
9 串口相关参数、函数
10
11 *****/
12
13 #define UART_BAUD_RATE 9600 /* 串口速率是9600bps */
14 #define UART_BASE 0x10000000
15 #define UART_IC_REG 0x00000003 /* Line Control Register */
16 #define UART_IE_REG 0x00000001 /* Interrupt Enable Register */
17 #define UART_TH_REG 0x00000000 /* Transmitter Holding Register */
18 #define UART_LS_REG 0x00000005 /* Line Status Register */
19 #define UART_DLB1_REG 0x00000000 /* Divisor Latch Byte 1(LSB) */
20 #define UART_DLB2_REG 0x00000001 /* Divisor Latch Byte 2(MSB) */
```

编译如下：



```
mxlb@mxlb-virtual-machine: ~/Chapter15/ucosii_OpenMIPS
make[1]: 离开目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/port"
make[1]: 进入目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/common"
mips-sde-elf-gcc -I/home/mxlb/Chapter15/ucosii_OpenMIPS/include -I/home/mxlb/Chapter15/ucosii_OpenMIPS/ucos -I/home/mxlb/Chapter15/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o openmips.o openmips.c
openmips.c:144: warning: return type of 'main' is not 'int'
mips-sde-elf-ld -r -o common.o openmips.o
make[1]: 离开目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/common"
make[1]: 进入目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/ucos"
make[1]: 对"all"无需做任何事。
make[1]: 离开目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/ucos"
make[1]: 进入目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/port"
Ubuntu Software : -I/home/mxlb/Chapter15/ucosii_OpenMIPS/include -I/home/mxlb/Chapter15/ucosii_OpenMIPS/ucos -I/home/mxlb/Chapter15/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o os_cpu_c.o os_cpu_c.c
os_cpu_c.c: In function 'BSP_Exception_Handler':
os_cpu_c.c:372: warning: unused variable 'EPC'
mips-sde-elf-gcc -I/home/mxlb/Chapter15/ucosii_OpenMIPS/include -I/home/mxlb/Chapter15/ucosii_OpenMIPS/ucos -I/home/mxlb/Chapter15/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o os_cpu_a.o os_cpu_a.S
mips-sde-elf-ld -r -o port.o os_cpu_c.o os_cpu_a.o
make[1]: 离开目录"/home/mxlb/Chapter15/ucosii_OpenMIPS/port"
mips-sde-elf-gcc -Tram.ld -o ucosii.om common/common.o ucus/ucos.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary ucosii.om ucosii.bin
mips-sde-elf-obidump -D ucosii.om > ucosii.asm
./BinMerge.exe -f ucosii.bin -o OS.bin
mxlb@mxlb-virtual-machine: ~/Chapter15/ucosii_OpenMIPS$
```

得到二进制文件，将 OS.bin 写入板子上的 Flash，当 OpenMIPS 运行时，会首先运行 BootLoader，后者将 μ C/OC-的代码复制到 SDRAM 中，然后跳转到 SDRAM，把控制权交给 μ C/OS-II，于是 μ C/OS-II 就运行起来了。

5.10 下板验证



除了 SDRAM 阶段配置的 MIGIP 核，还需要配置 Clocking Wizard IP 核。这里直接在 Vivado Tcl Console 里执行：

```
create_ip -name clk_wiz -vendor xilinx.com -library ip -version 6.0
-module_name clk_wiz_0
set_property -dict [list \
    CONFIG.PRIMITIVE {MMCM} \
    CONFIG.PRIM_IN_FREQ {100.000} \
    CONFIG.PRIM_SOURCE {Single_ended_clock_capable_pin} \
```

```

CONFIG.NUM_OUT_CLKS {2} \
CONFIG.CLKOUT1_USED {true} \
CONFIG.CLKOUT2_USED {true} \
CONFIG.CLKOUT1_REQUESTED_OUT_FREQ {100.000} \
CONFIG.CLKOUT2_REQUESTED_OUT_FREQ {200.000} \
CONFIG.CLKOUT1_REQUESTED_PHASE {0.000} \
CONFIG.CLKOUT2_REQUESTED_PHASE {0.000} \
CONFIG.CLKOUT1_REQUESTED_DUTY_CYCLE {50.000} \
CONFIG.CLKOUT2_REQUESTED_DUTY_CYCLE {50.000} \
CONFIG.USE_LOCKED {true} \
CONFIG.USE_RESET {false} \
CONFIG.USE_PHASE_ALIGNMENT {true} \
] [get_ips clk_wiz_0]
generate_target all [get_ips clk_wiz_0]

```

并配置 xdc 文件如下:

```

## Clock signal
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports clk_in]

## Switches
set_property -dict {PACKAGE_PIN J15 IOSTANDARD LVCMOS33} [get_ports flash_continue]
set_property -dict {PACKAGE_PIN L16 IOSTANDARD LVCMOS33} [get_ports {gpio_i[1]}]
set_property -dict {PACKAGE_PIN M13 IOSTANDARD LVCMOS33} [get_ports {gpio_i[2]}]
set_property -dict {PACKAGE_PIN R15 IOSTANDARD LVCMOS33} [get_ports {gpio_i[3]}]
set_property -dict {PACKAGE_PIN R17 IOSTANDARD LVCMOS33} [get_ports {gpio_i[4]}]
set_property -dict {PACKAGE_PIN T18 IOSTANDARD LVCMOS33} [get_ports {gpio_i[5]}]
set_property -dict {PACKAGE_PIN U18 IOSTANDARD LVCMOS33} [get_ports {gpio_i[6]}]
set_property -dict {PACKAGE_PIN R13 IOSTANDARD LVCMOS33} [get_ports {gpio_i[7]}]
set_property -dict {PACKAGE_PIN T8 IOSTANDARD LVCMOS18} [get_ports {gpio_i[8]}]
set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS18} [get_ports {gpio_i[9]}]
set_property -dict {PACKAGE_PIN R16 IOSTANDARD LVCMOS33} [get_ports {gpio_i[10]}]
set_property -dict {PACKAGE_PIN T13 IOSTANDARD LVCMOS33} [get_ports {gpio_i[11]}]
set_property -dict {PACKAGE_PIN H6 IOSTANDARD LVCMOS33} [get_ports {gpio_i[12]}]
set_property -dict {PACKAGE_PIN U12 IOSTANDARD LVCMOS33} [get_ports {gpio_i[13]}]
set_property -dict {PACKAGE_PIN U11 IOSTANDARD LVCMOS33} [get_ports {gpio_i[14]}]
set_property -dict {PACKAGE_PIN V10 IOSTANDARD LVCMOS33} [get_ports {gpio_i[15]}]

```

```

## LEDs
set_property -dict { PACKAGE_PIN H17    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[16] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
set_property -dict { PACKAGE_PIN K15    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[17] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
set_property -dict { PACKAGE_PIN J13    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[18] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
set_property -dict { PACKAGE_PIN N14    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[19] }];
#IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN R18    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[20] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
set_property -dict { PACKAGE_PIN V17    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[21] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
set_property -dict { PACKAGE_PIN U17    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[22] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
set_property -dict { PACKAGE_PIN U16    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[23] }];
#IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN V16    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[24] }];
#IO_L16N_T2_A15_D31_14 Sch=led[8]
set_property -dict { PACKAGE_PIN T15    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[25] }];
#IO_L14N_T2_SRCC_14 Sch=led[9]
set_property -dict { PACKAGE_PIN U14    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[26] }];
#IO_L22P_T3_A05_D21_14 Sch=led[10]
set_property -dict { PACKAGE_PIN T16    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[27] }];
#IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
set_property -dict { PACKAGE_PIN V15    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[28] }];
#IO_L16P_T2_CSI_B_14 Sch=led[12]
set_property -dict { PACKAGE_PIN V14    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[29] }];
#IO_L22N_T3_A04_D20_14 Sch=led[13]
set_property -dict { PACKAGE_PIN V12    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[30] }];
#IO_L20N_T3_A07_D23_14 Sch=led[14]
set_property -dict { PACKAGE_PIN V11    IOSTANDARD LVCMOS33 } [get_ports { gpio_o[31] }];
#IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]

## SEG
set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[15] }]
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[14] }]
set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[13] }]
set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[12] }]
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[11] }]
set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[10] }]
set_property -dict { PACKAGE_PIN R10 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[9] }]
set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[8] }]
set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { gpio_o[7] }]

```

```

set_property -dict {PACKAGE_PIN K2 IOSTANDARD LVCMOS33} [get_ports {gpio_o[6]]}
set_property -dict {PACKAGE_PIN T14 IOSTANDARD LVCMOS33} [get_ports {gpio_o[5]]}
set_property -dict {PACKAGE_PIN P14 IOSTANDARD LVCMOS33} [get_ports {gpio_o[4]]}
set_property -dict {PACKAGE_PIN J14 IOSTANDARD LVCMOS33} [get_ports {gpio_o[3]]}
set_property -dict {PACKAGE_PIN T9 IOSTANDARD LVCMOS33} [get_ports {gpio_o[2]]}
set_property -dict {PACKAGE_PIN J18 IOSTANDARD LVCMOS33} [get_ports {gpio_o[1]]}
set_property -dict {PACKAGE_PIN J17 IOSTANDARD LVCMOS33} [get_ports {gpio_o[0]]}

## Buttons
set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports rst_n]
set_property -dict {PACKAGE_PIN N17 IOSTANDARD LVCMOS33} [get_ports {gpio_i[0]]}

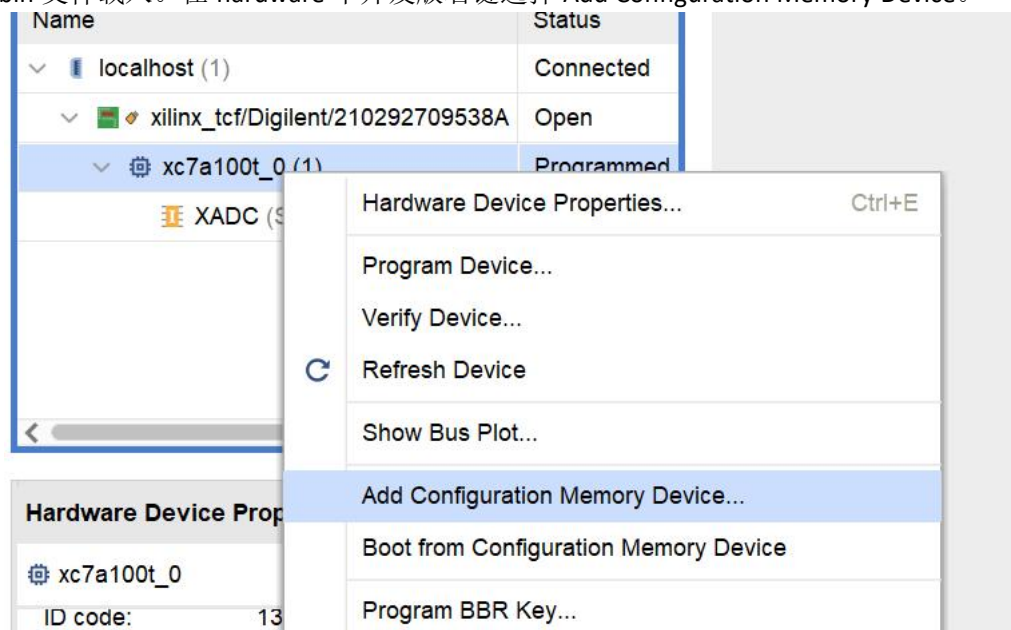
## USB-RS232 Interface
set_property -dict {PACKAGE_PIN C4 IOSTANDARD LVCMOS33} [get_ports uart_in]
set_property -dict {PACKAGE_PIN D4 IOSTANDARD LVCMOS33} [get_ports uart_out]

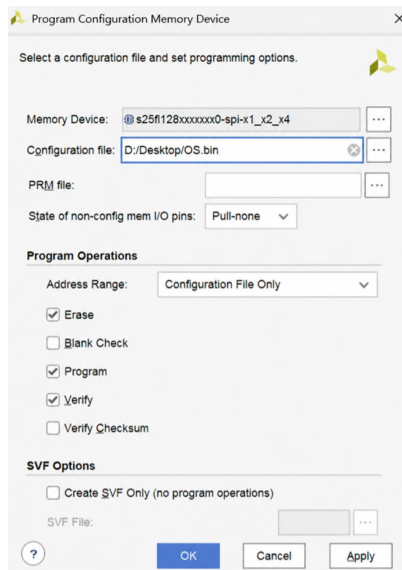
## Quad SPI Flash
set_property -dict {PACKAGE_PIN K17 IOSTANDARD LVCMOS33} [get_ports sdo]
set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports sdi]
set_property -dict {PACKAGE_PIN L14 IOSTANDARD LVCMOS33} [get_ports wp_n]
set_property -dict {PACKAGE_PIN M14 IOSTANDARD LVCMOS33} [get_ports hld_n]
set_property -dict {PACKAGE_PIN L13 IOSTANDARD LVCMOS33} [get_ports cs_n]

set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets rst_n_IBUF]

```

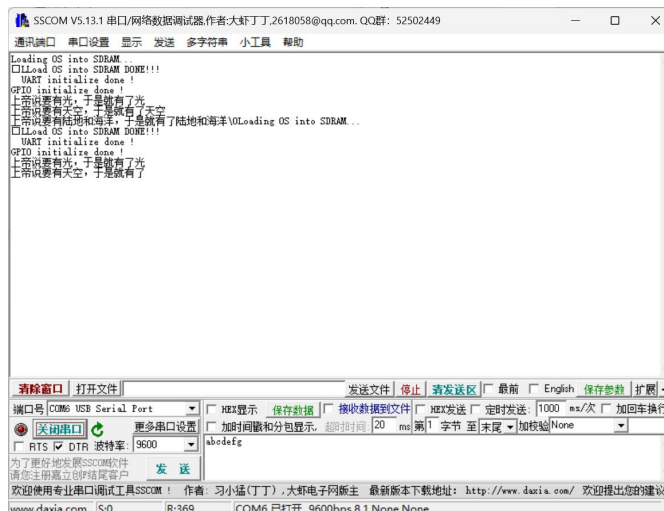
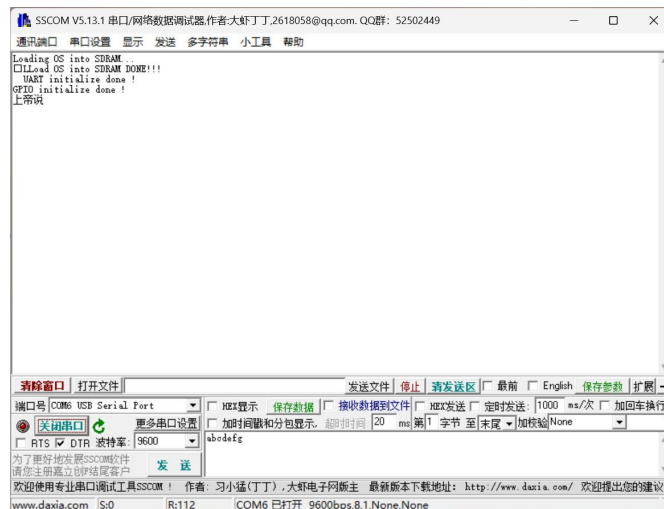
综合、实现、生成比特流完成之后，在下板之前还需要把之前在 Ubuntu 中编译好的 OS.bin 文件载入。在 hardware 中开发板右键选择 Add Configuration Memory Device。

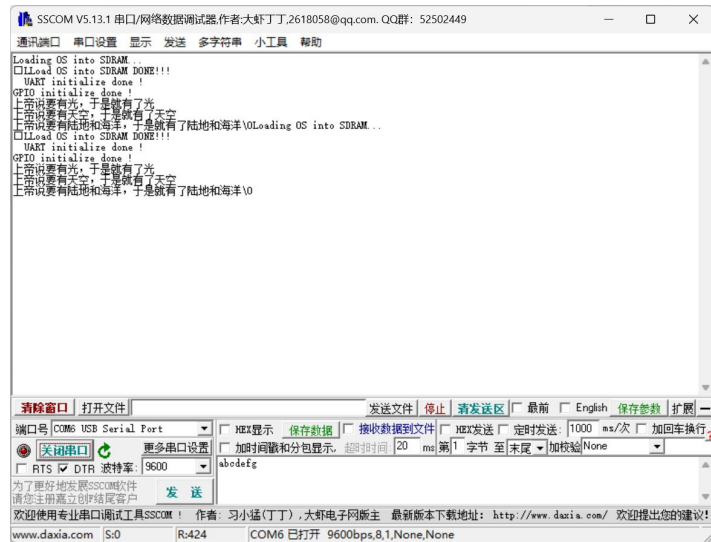




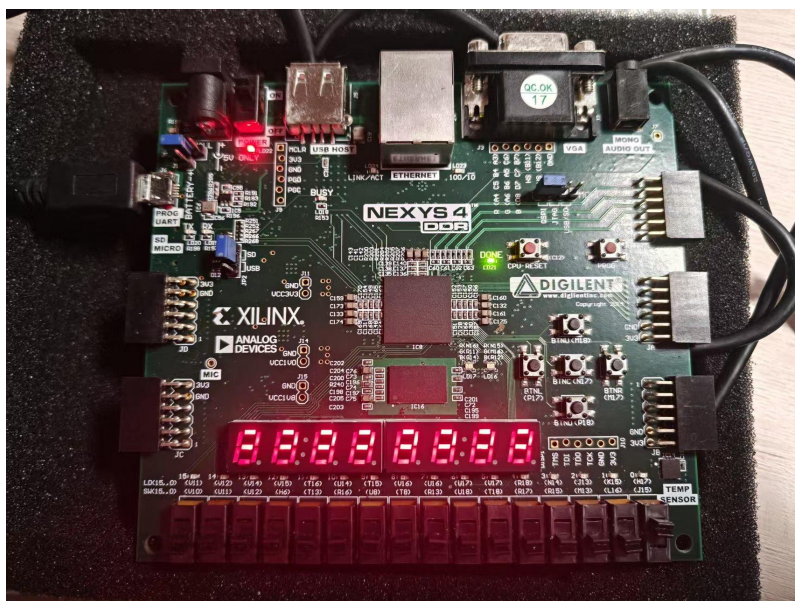
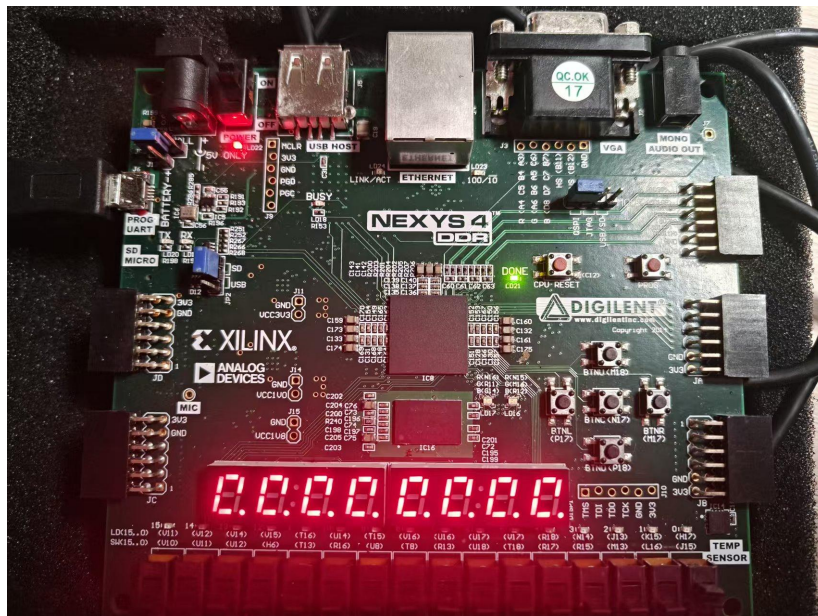
下板后，SSCOM V5.13.1 串口调试器选择对应的端口号 COM6 USB Serial Port，调整波特率为 9600，取消加时间戳和分包显示，点击打开串口，将开发版右侧第一个开关上拨，并按下 C12 对应的 CPU RESET 按钮，即可看到实验结果。

串口程序：汉字每隔 100ms 显示一个





下板结果：7 段数码管的显示 100ms 变化一次



六、实验总结

在本次 μ C/OS-II 系统移植实验中，我基于实验一所实现的 89 条指令 MIPS CPU 核心，通过引入 Wishbone 总线协议，成功构建了一个具备完整软硬件协同工作能力的 SOPC 系统。在此基础上，通过搭建 Linux 交叉编译环境，完成了操作系统的适配、编译与下板验证。

观察到该汇编代码中有许多值为 1 的 label，为了防止错误并且增强程序的可读性，将这些 label 分别改为 L1、L2、L3。本实验设计的串口波特率为 9600，UART 控制器的时钟频率为 100MHZ，因此分频系数应设置为：28BH。需要 boostloader 汇编程序配置 UART 分频系数寄存器为 28BH。

本实验设计的串口波特率为 9600，UART 控制器的时钟频率为 100MHZ，因此分频系数应设置为：28BH。修改内容位于 openmips.h 文件内需要修改系统时钟频率为 100000000。因为该文件内定义了波特率为 9600，因此应用程序会计算出正确的分频系数。

本次实验不同于以往，打破了学科之间的壁垒，将计算机组成原理、计算机系统结构、操作系统等核心课程的理论知识融合在一起。我亲身体会到了从底层的指令集扩展、硬件总线仲裁、控制器时序设计，到软件层面的 BootLoader 编写、C 语言交叉编译，再到操作系统内核在裸机上跑起来的完整软硬件协同设计流程。

通过引入 Wishbone 规范及 WB_CONMAX 互连矩阵，我深入理解了总线在片上系统中所扮演的“桥梁”和“枢纽”角色。主设备与从设备之间通过简明规范的握手信号（如 cyc、stb、ack）进行解耦通信，这极大地降低了系统模块间的耦合度，为外设的灵活扩展打下了良好的工程基础。

在软件移植阶段，我不仅理清了 μ C/OS-II 的代码结构，还深入理解了 BootLoader 的工作原理。系统上电后，由于 Flash 速度较慢且仅作只读存储，需由 BootLoader 程序将操作系统的二进制镜像（OS.bin）从 SPI Flash 搬运至大容量的高速 DDR2 SDRAM 中，然后再跳转至 SDRAM 执行，从而完成系统的初始化和控制权递交。

在首次下板进行 SSCOM 调试时，串口接收区出现无法识别的乱码。经排查，由于硬件 SOPC 的主频配置为了 100MHz（利用 Clocking Wizard 进行 2 分频产生），而软件工程配置文件 openmips.h 中针对 IN_CLK 的宏定义与 BootLoader 的波特率计算未保持一致。在将软硬件中的主频参数统一校正为 100MHz 并重新编译 BootLoader 后，串口数据输出恢复正常。